

Universidade Federal do Cariri
Ciência da Computação
Lógica

**Chaos Sudoku: Formalização em Lógica
Proposicional e Verificação Experimental
com SAT Solver**

Alunos: David Hudson Gomes Alves e Vitória do Nascimento Pontes
Professor: Luis Henrique Bustamante de Moraes

Agosto
2026

Conteúdo

1	Introdução	1
2	Descrição do problema e das instâncias	1
2.1	Representação JSON	1
2.2	Diferença em relação ao Sudoku tradicional	2
2.3	Instâncias utilizadas	2
3	Formalização matemática	2
3.1	Variáveis proposicionais	2
3.2	Codificação dos identificadores DIMACS	3
3.3	Família F1: pelo menos um valor	3
3.4	Família F2: no máximo um valor por célula	3
3.5	Família F3: unicidade nas linhas	3
3.6	Família F4: unicidade nas colunas	4
3.7	Família F5: unicidade nas regiões	4
3.8	Família F6: pistas	4
3.9	Contagem total	4
4	Implementação do gerador DIMACS	5
4.1	Leitura e validação da instância	5
4.2	Organização interna das cláusulas	5
4.3	Geração do DIMACS	5
5	Experimentação com o SAT Solver	6
5.1	Instância exemplo_4	6
5.2	Instância exemplo_4_unsat	6
5.3	Instância exemplo_5	7
5.4	Instância exemplo_6	7
5.5	Instância exemplo_7	7
5.6	Instância exemplo_8	7
5.7	Instância exemplo_9	8
5.8	Reconstrução do modelo SAT	8
5.9	Observação sobre os tempos	8
6	Análise dos resultados	8
6.1	Crescimento do número de variáveis	8
6.2	Crescimento do número de cláusulas	9
6.3	Interpretação dos casos UNSAT	9
6.4	Adequação da formalização	9

7	Conclusão	10
8	Repositório	10

1 Introdução

O trabalho proposto tem como finalidade aproximar a teoria da Lógica Proposicional de uma aplicação computacional concreta. O enunciado solicita a escolha de um problema combinatório, sua formalização em lógica proposicional, a geração de instâncias no formato DIMACS e a utilização de um SAT solver moderno para verificar experimentalmente a satisfatibilidade.

Neste projeto foi escolhido o *Chaos Sudoku*, uma variação do Sudoku na qual as regiões podem ser irregulares. Cada possível colocação de um valor em uma célula pode ser tratada como uma proposição, e as regras do jogo podem ser expressas por restrições lógicas. A utilização de SAT é particularmente interessante porque o solver não precisa conhecer as regras específicas do Sudoku. Ele recebe somente uma fórmula CNF. Assim, o conhecimento do domínio fica concentrado na etapa de modelagem, enquanto o processo de busca por uma atribuição satisfatória é delegado ao solver.

2 Descrição do problema e das instâncias

Uma instância de Chaos Sudoku é composta por uma grade quadrada $N \times N$. Cada célula deve receber um inteiro entre 1 e N . Além da restrição de preenchimento, cada valor deve aparecer exatamente uma vez em cada linha, exatamente uma vez em cada coluna e exatamente uma vez em cada região irregular. As regiões são fornecidas explicitamente por uma matriz $N \times N$. O identificador presente em cada posição informa a região à qual aquela célula pertence. Para uma instância válida, existem exatamente N regiões, identificadas por $1, \dots, N$, e cada região possui exatamente N células.

2.1 Representação JSON

O projeto representa cada instância por duas matrizes: regiões e pistas. Um exemplo 4×4 é:

```
{
  "regioes": [
    [3, 1, 1, 1],
    [3, 3, 3, 1],
    [2, 4, 4, 4],
    [2, 2, 2, 4]
  ],
  "pistas": [
    [1, 0, 0, 0],
```

```

    [0, 3, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 3]
  ]
}
```

Na matriz de pistas, o valor zero significa que a célula está inicialmente vazia. Um valor entre 1 e N fixa aquele valor como parte da instância.

2.2 Diferença em relação ao Sudoku tradicional

No Sudoku tradicional, as regiões normalmente possuem uma forma regular e sua geometria pode ser inferida a partir de N . No Chaos Sudoku, essa hipótese não é feita. A matriz regiões é parte da entrada e permite descrever regiões irregulares. Isso torna a formalização mais geral: a mesma lógica de unicidade é aplicada ao conjunto de células que compartilham um identificador de região.

2.3 Instâncias utilizadas

O repositório contém sete fórmulas CNF correspondentes às instâncias `exemplo_4`, `exemplo_4_unsat`, `exemplo_5`, `exemplo_6`, `exemplo_7`, `exemplo_8` e `exemplo_9`. Os tamanhos crescem de 4×4 até 9×9 , permitindo observar o aumento da representação.

3 Formalização matemática

3.1 Variáveis proposicionais

Seja N o tamanho da grade. Definimos o conjunto de variáveis:

$$X = \{x_{ijv} \mid 1 \leq i, j, v \leq N\}.$$

A interpretação é direta: x_{ijv} é verdadeira se, e somente se, a célula localizada na linha i e coluna j contém o valor v . Para uma célula existem N possibilidades e, portanto, são criadas N variáveis.

Como existem N^2 células, o número total de variáveis é:

$$|X| = N^3.$$

Essa escolha é conhecida como codificação *one-hot*: em vez de representar diretamente o número escrito na célula, representamos separadamente cada possibilidade de valor.

3.2 Codificação dos identificadores DIMACS

O DIMACS utiliza inteiros positivos para representar variáveis. O projeto associa cada x_{ijv} ao identificador:

$$\text{var}(i, j, v) = N^2(i - 1) + N(j - 1) + v.$$

A primeira parcela pula todas as linhas anteriores; a segunda pula as células anteriores da linha atual; a terceira seleciona o valor dentro da célula. O maior identificador possível é N^3 , exatamente a quantidade de variáveis.

Por exemplo, para $N = 4$, a célula $(1, 1)$ usa os IDs 1, 2, 3, 4; a célula $(1, 2)$ usa 5, 6, 7, 8; e a célula $(2, 1)$ começa no ID 17. Isso permite converter a estrutura tridimensional dos índices (i, j, v) para a numeração linear exigida pelo DIMACS.

3.3 Família F1: pelo menos um valor

Para cada célula (i, j) , é necessário garantir que algum valor seja escolhido:

$$(x_{ij1} \vee x_{ij2} \vee \cdots \vee x_{ijN}).$$

Existe uma cláusula desse tipo para cada uma das N^2 células. Logo, F1 possui N^2 cláusulas. Essa família, isoladamente, permite que mais de um valor seja escolhido na mesma célula; por isso é necessária a família seguinte.

3.4 Família F2: no máximo um valor por célula

Para impedir que uma célula contenha dois valores simultaneamente, para cada par de valores $v_1 < v_2$ incluímos:

$$(\neg x_{ijv_1} \vee \neg x_{ijv_2}).$$

Para cada célula existem $\binom{N}{2}$ pares. Como existem N^2 células, a quantidade de cláusulas de F2 é $N^2 \cdot \binom{N}{2}$. Com F1 e F2 em conjunto, cada célula recebe exatamente um valor.

3.5 Família F3: unicidade nas linhas

Para cada linha i e valor v , o mesmo valor não pode aparecer em duas colunas distintas. Para cada par $j_1 < j_2$:

$$(\neg x_{ij_1v} \vee \neg x_{ij_2v}).$$

Há N linhas, N valores e $\binom{N}{2}$ pares de colunas, totalizando $N^2 \cdot \binom{N}{2}$ cláusulas.

3.6 Família F4: unicidade nas colunas

Analogamente, para cada coluna j e valor v , duas linhas diferentes não podem receber o mesmo valor. Para $i_1 < i_2$:

$$(\neg x_{i_1 j v} \vee \neg x_{i_2 j v}).$$

Também são geradas $N^2 \cdot \binom{N}{2}$ cláusulas.

3.7 Família F5: unicidade nas regiões

Se duas células pertencem à mesma região r , elas não podem conter o mesmo valor. Para cada região, valor e par de células distintas da região:

$$(\neg x_{i_1 j_1 v} \vee \neg x_{i_2 j_2 v}).$$

Como cada instância válida possui N regiões e cada região contém exatamente N células, cada região também gera $\binom{N}{2}$ pares para cada valor. Portanto, F5 possui $N^2 \cdot \binom{N}{2}$ cláusulas.

3.8 Família F6: pistas

Uma pista $(i, j) = v$ é representada por uma cláusula unitária:

$$x_{ijv}.$$

Se a instância possui G pistas, F6 acrescenta exatamente G cláusulas. Uma pista contraditória não precisa ser detectada pelo gerador: ela pode ser representada normalmente e fará com que a fórmula seja UNSAT.

3.9 Contagem total

Somando as famílias:

$$M = N^2 + 4N^2 \cdot \binom{N}{2} + G.$$

Como $\binom{N}{2} = \frac{N(N-1)}{2}$, obtemos:

$$M = N^2(1 + 2N(N-1)) + G.$$

Assim, a fórmula possui N^3 variáveis e M cláusulas. O termo dominante cresce proporcionalmente a N^4 .

4 Implementação do gerador DIMACS

O gerador está implementado em `src/gerador.py` e utiliza somente recursos da biblioteca padrão do Python. A interface recebe dois argumentos principais: o tamanho N e o caminho do arquivo JSON.

```
python3 src/gerador.py N ARQUIVO_JSON
```

A saída é produzida na saída padrão. Isso permite redirecionar diretamente o resultado para um arquivo `.cnf`.

```
python3 src/gerador.py 4 instances/exemplo_4.json > exemplo_4.cnf
```

4.1 Leitura e validação da instância

A função `ler_instancia` abre o JSON e delega a validação estrutural para `validar_instancia`. O gerador verifica a existência do campo `regioes`, o formato $N \times N$ das matrizes, os identificadores de regiões e a quantidade de células por região. O campo `pistas` é opcional; quando não existe, todas as células são tratadas como vazias.

É importante observar que essas verificações são estruturais. O programa não tenta decidir previamente se as pistas são compatíveis com uma solução. Essa decisão é justamente uma das funções do SAT solver.

4.2 Organização interna das cláusulas

Cada família de restrição é construída por uma função específica. As cláusulas são representadas internamente por listas de inteiros, em que um inteiro positivo representa uma variável e um inteiro negativo representa sua negação.

```
def var(i, j, v, n):  
    return n**2 * (i - 1) + n * (j - 1) + v
```

Para gerar pares sem repetição, o código utiliza `combinations(..., 2)`, evitando a criação duplicada de uma mesma cláusula. A estrutura de regiões é obtida a partir da matriz fornecida pela instância.

4.3 Geração do DIMACS

O cabeçalho DIMACS segue o formato `'p cnf N M'`. No primeiro exemplo, o arquivo começa com `'p cnf 64 404'`: 64 representa N^3 para $N = 4$ e 404 corresponde às 400 cláusulas estruturais mais quatro pistas.

Cada cláusula é encerrada pelo literal 0. Por exemplo, a cláusula de pelo menos um valor para a primeira célula de uma instância 4×4 é escrita como:

5 Experimentação com o SAT Solver

A etapa experimental utiliza o CaDiCaL, conforme recomendado no enunciado da disciplina. O objetivo é fornecer ao solver as fórmulas CNF geradas e registrar o resultado de satisfatibilidade, o tamanho da fórmula e o tempo de execução. Foram utilizadas sete instâncias, cobrindo tamanhos de 4×4 , 5×5 , 6×6 , 7×7 , 8×8 e 9×9 . Os arquivos CNF presentes no projeto registram os seguintes cabeçalhos:

Tabela 1: Resultados experimentais com CaDiCaL.

Instância	N	Variáveis	Cláusulas	Pistas	Resultado	Tempo (s)
exemplo_4	4	64	404	4	SAT	0,013
exemplo_4_unsat	4	64	402	2	UNSAT	0,001
exemplo_5	5	125	1027	2	UNSAT	0,002
exemplo_6	6	216	2202	6	SAT	0,003
exemplo_7	7	343	4167	2	UNSAT	0,002
exemplo_8	8	512	7240	8	SAT	0,003
exemplo_9	9	729	11754	9	SAT	0,003

5.1 Instância exemplo_4

A instância 4×4 possui 64 variáveis, 404 cláusulas e quatro pistas. O resultado é SAT. Uma solução reconstruída a partir do modelo é:

1	4	3	2
4	3	2	1
3	2	1	4
2	1	4	3

Essa grade satisfaz as quatro condições do problema: cada linha contém 1, 2, 3, 4 sem repetição; o mesmo ocorre nas colunas; e cada região contém os quatro valores.

5.2 Instância exemplo_4_unsat

Essa instância mantém as mesmas regiões, mas possui duas pistas iguais a 1 na primeira linha. As pistas obrigam simultaneamente x_{111} e x_{121} a serem verdadeiras. Entretanto, F3 contém a cláusula $(\neg x_{111} \vee \neg x_{121})$, produzindo uma contradição direta. O solver, portanto, retorna UNSATISFIABLE.

5.3 Instância exemplo_5

A instância 5×5 possui 125 variáveis e 1.027 cláusulas. Ela contém duas pistas iguais a 1 na primeira linha. Pela mesma razão da instância 4×4 insatisfatível, as pistas conflitam com a restrição de unicidade por linha. O resultado registrado é UNSAT.

5.4 Instância exemplo_6

A instância 6×6 possui 216 variáveis, 2.202 cláusulas e seis pistas. As regiões são seis blocos regulares de 3×2 células. O resultado é SAT. A solução reconstruída é:

1	6	5	3	4	2
3	2	4	6	1	5
6	4	3	5	2	1
5	1	2	4	6	3
2	3	6	1	5	4
4	5	1	2	3	6

5.5 Instância exemplo_7

A instância 7×7 possui 343 variáveis e 4.167 cláusulas. Assim como `exemplo_4_unsat` e `exemplo_5`, suas duas primeiras pistas são 1 na mesma linha, tornando a fórmula insatisfatível. O solver retorna UNSAT.

5.6 Instância exemplo_8

A instância 8×8 possui 512 variáveis e 7.240 cláusulas, com oito pistas. As regiões formam oito blocos de 4×2 células. O resultado é SAT e uma solução reconstruída é:

1	3	7	5	6	8	4	2
8	2	4	6	7	3	5	1
7	6	3	2	8	4	1	5
5	8	1	4	2	7	3	6
6	7	8	3	5	1	2	4
2	4	5	1	3	6	8	7
4	5	6	8	1	2	7	3
3	1	2	7	4	5	6	8

5.7 Instância exemplo_9

A maior instância utilizada possui $N = 9$, 729 variáveis e 11.754 cláusulas. As regiões são nove blocos de 3×3 células e existem nove pistas na primeira linha, uma para cada valor de 1 a 9. O resultado é SAT. A grade reconstruída é:

1	2	3	4	5	6	7	8	9
8	7	5	3	9	1	2	4	6
6	9	4	8	2	7	1	3	5
7	6	8	2	1	5	3	9	4
5	3	1	6	4	9	8	2	7
2	4	9	7	3	8	5	6	1
4	5	6	1	8	2	9	7	3
9	8	7	5	6	3	4	1	2
3	1	2	9	7	4	6	5	8

5.8 Reconstrução do modelo SAT

A reconstrução é feita interpretando os literais positivos presentes no modelo. Se o solver informa que a variável correspondente a x_{ijv} é verdadeira, o algoritmo coloca v na posição (i, j) . As famílias F1 e F2 garantem que cada célula tenha exatamente um valor, evitando ambiguidades durante a reconstrução.

Essa etapa é importante porque demonstra que o solver não apenas classifica a fórmula. Em uma instância SAT, seu modelo contém informação suficiente para recuperar uma solução concreta do problema original.

5.9 Observação sobre os tempos

Os tempos medidos ficaram entre 1 e 13 milissegundos, sem relação direta com o tamanho da instância: a maior demora observada ocorreu na instância 4×4 (0,013 s), enquanto instâncias maiores, como a 9×9 , foram resolvidas em 3 milissegundos. Esse comportamento é esperado para fórmulas desse porte, pequenas o suficiente para que fatores como o tempo de inicialização do solver dominem sobre a dificuldade real de busca. Diferenças de desempenho mais claras entre as instâncias provavelmente só apareceriam em grades bem maiores que $N = 9$.

6 Análise dos resultados

6.1 Crescimento do número de variáveis

A quantidade de variáveis é N^3 . Isso significa que aumentar o tamanho da grade provoca crescimento relativamente rápido da representação. Por exemplo,

passar de $N = 4$ para $N = 9$ aumenta o número de variáveis de 64 para 729.

6.2 Crescimento do número de cláusulas

O número de cláusulas sem pistas é $N^2(1 + 2N(N - 1))$. O termo dominante é aproximadamente $2N^4$. Esse crescimento é mais rápido que o das variáveis porque as restrições de unicidade são codificadas comparando pares de possibilidades.

Tabela 2: Crescimento do número de variáveis e cláusulas.

N	N^3 variáveis	Cláusulas sem pistas	Exemplo no projeto
4	64	400	404
5	125	1025	1027
6	216	2196	2202
7	343	4165	4167
8	512	7232	7240
9	729	11745	11754

6.3 Interpretação dos casos UNSAT

Os casos UNSAT presentes no conjunto experimental são particularmente didáticos. A insatisfatibilidade não decorre de uma falha do gerador, mas da própria instância: duas pistas obrigam o mesmo valor a ocupar duas células da mesma linha. A formalização traduz essa regra em cláusulas e o solver detecta que elas não podem ser satisfeitas em conjunto.

6.4 Adequação da formalização

A codificação *one-hot* é adequada ao objetivo do trabalho porque torna a interpretação das variáveis muito simples. Cada variável corresponde diretamente a uma afirmação sobre uma célula. Além disso, as restrições de unicidade podem ser expressas com cláusulas binárias, facilitando a implementação e a explicação matemática.

Como desvantagem, a abordagem não é a mais econômica em número de cláusulas. Para cada conjunto de possibilidades são gerados pares, fazendo com que o tamanho da fórmula cresça aproximadamente como N^4 . Uma investigação futura poderia comparar outras codificações ou esquemas de cardinalidade.

7 Conclusão

O trabalho demonstrou como um problema combinatório pode ser transformado sistematicamente em uma instância de SAT. A partir de uma grade $N \times N$, foram definidas N^3 variáveis proposicionais, cada uma representando uma possível atribuição de valor a uma célula. As regras do Chaos Sudoku foram então decompostas em famílias de cláusulas CNF, permitindo uma correspondência clara entre a especificação do problema e a fórmula lógica.

A implementação em Python materializa essa formalização em arquivos DIMACS. A representação de regiões por uma matriz torna o gerador capaz de trabalhar com regiões irregulares, característica central do Chaos Sudoku. A interface por linha de comando também permite integrar o gerador diretamente a um SAT solver.

Os experimentos com sete instâncias mostraram resultados SAT e UNSAT. Nos casos satisfativos, foi possível recuperar as grades a partir dos modelos; nos casos insatisfativos, a causa pode ser explicada pelas pistas conflitantes e pelas cláusulas de unicidade. O crescimento de 64 para 729 variáveis e de 404 para 11.754 cláusulas evidencia como a representação proposicional aumenta com N , embora as instâncias avaliadas continuem pequenas o suficiente para serem resolvidas muito rapidamente.

Dessa forma, a experiência evidencia a principal vantagem da abordagem SAT: depois de formalizado, o problema passa a ser tratado por um mecanismo geral de decisão. A qualidade do resultado depende, entretanto, da escolha da modelagem. Uma continuação natural deste projeto seria comparar a codificação adotada com uma representação alternativa, avaliar outros solvers e estudar técnicas para reduzir o número de cláusulas.

8 Repositório

Projeto Chaos Sudoku. Repositório: <https://github.com/VitoriaPontes/chaos-sudoku>